

****AI pen-test weapon unleashed**** The campaign ran roughly ten days, targeting multiple South Korean financial firms. Are we ready to defend hardware supply chains against AI-driven pen-test abuse?

****⚠ INCIDENT OVERVIEW**** CrowdStrike Intelligence uncovered a coordinated effort that leveraged the ****ARTEX**** AI pentesting framework to bypass traditional perimeter controls. By automating vulnerability discovery and chaining exploits, threat actors harvested credentials and exfiltrated sensitive transaction logs. The attack surface was not a web app—it was the firmware update pipeline of on-premise banking appliances. Once a malicious firmware

image was signed with a compromised signing key, the

HARDWARE & SUPPLY-CHAIN IMPACT What most teams assume: "If the silicon is vetted, the software layer is safe." What actually happens: adversaries weaponize **AI-driven pen-testing** to locate firmware signing weaknesses, then inject malicious payloads that survive OS reinstall. In this case, the compromised **nLoadImage** hook in **clr.dll**—a technique normally used by EDRs—was subverted to hide the payload in memory, evading endpoint telemetry. The result? Full-disk encryption keys leaked

Last week my red-team exercised a similar vector on a client's ATM network. We slipped a rogue image through their OTA update process, and the device dutifully executed our payload while reporting a clean status. The lesson hit hard: hardware integrity checks must be immutable, and every firmware signature must be anchored to a hardware-rooted trust anchor.

****[HARD ON] OPERATIONAL CHECKLIST****

- Enforce ****Secure Boot**** with immutable keys stored in TPM.
- Deploy continuous ****firmware integrity monitoring**** that hashes each image against a trusted baseline.
- Integrate ****hardware provenance**** data into your SIEM to flag unsigned or out-of-band updates.
- Conduct regular ****hardware-level red-team drills**** using tools like ****CLR-Unhook**** to validate that hooking mechanisms cannot be hijacked.
- Audit supply-chain contracts for

Tool Recommendation --- **CLR-Unhook** (**214**) – This open-source utility strips the `nLoadImage` hook from `clr.dll`, allowing analysts to view the original .NET assembly before security products intervene. In environments where firmware loaders rely on .NET components, **CLR-Unhook** helps expose hidden malicious code that would otherwise be masked by EDR hooks. Deploy it in a sandboxed analysis pipeline to verify that no stealthy in-memory modifications survive the unhook

In 12 months, any organization that continues to trust unsigned firmware updates will see a breach cascade across its supply chain. **Would you rather secure every hardware touchpoint now, or scramble after the next AI-augmented pen-test weapon hits the production floor?** | Source: <https://thehackernews.com/2026/10/artex-ai-pentesting-tool-use> | #SupplyChainSecurity #HardwareTrust #DFIR